Rule-Based Room Cleaning Agent

CS0065 Intelligent Systems | Technical Assessment 2

What the program does

The agent follows simple condition-based rules in a two-room environment. It cleans the room it occupies when that room is dirty. When the current room is clean, it moves to a dirty room. If every room is clean, it becomes idle.

The program asks whether to run the required two-room task or the optional three-room extension, collects each initial room state, and asks for the step limit. It reports the current location, action, room grid, and environment dictionary after every step.

The extension chooses randomly among dirty rooms when more than one destination is available. The fixed random seed makes the example run repeatable. The program uses only Python standard-library features.

Sample run

```
Rule-Based Room Cleaning Agent
Number of rooms (2 or 3) [2]: 3
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean
Is room C Dirty or Clean? dirty
How many steps should the agent run? 5

Initial environment:
[A-Agent] [B-Clean] [C-Dirty]
Environment state: {'A': 'Dirty', 'B': 'Clean', 'C': 'Dirty'}

Step 1: Agent is in room A
Room A has been cleaned.
[A-Agent] [B-Clean] [C-Dirty]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Dirty'}

Step 2: Agent is in room A
Moving to room C.
[A-Clean] [B-Clean] [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Dirty'}

Step 3: Agent is in room C
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

Step 4: Agent is in room C
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Environment state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

Simulation complete.
Final location: room C
Final environment: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
```

Analysis questions

1. What rules does the agent follow?

If its current room is Dirty, it cleans that room. If the current room is Clean and another room is Dirty, it moves to a dirty room. When no dirty rooms remain, it stays in place and reports that it is idle.

2. How does the agent decide when to clean and when to move?

It checks the condition of the room it currently occupies. Dirty means clean; Clean means look for a different dirty room and move there. It does not move away from a room that is dirty before cleaning it.

3. What happens if both rooms are already clean at the start?

The agent starts in room A, finds it clean, and finds no dirty destination. It reports that all rooms are clean and becomes idle without changing either room.

4. How would you modify the agent to handle more than two rooms?

Store room names and conditions in a dictionary or list, and represent connections as a graph if rooms are not in a line. When the current room is clean, find reachable dirty rooms and choose a destination. The included three-room extension uses a dictionary and randomly selects among other dirty rooms.

Run with: python TA2_Santos.py

Python source code

```python
"""Two-room rule-based cleaning agent (with optional three-room extension).
Run: python TA2_Santos.py
No external packages are required.
"""
import random


class CleaningAgent:
    """Perceive the current room and clean it or move to a dirty room."""

    def __init__(self, environment, location="A"):
        self.environment = environment
        self.location = location

    def move(self, destination):
        """Move to another room in the environment."""
        if destination not in self.environment:
            raise ValueError(f"Unknown room: {destination}")
        self.location = destination

    def perceive_and_act(self):
        """Apply the rules once and return a description of the action."""
        if self.environment[self.location] == "Dirty":
            self.environment[self.location] = "Clean"
            return f"Room {self.location} has been cleaned."

        dirty_rooms = [
            room for room, condition in self.environment.items()
            if condition == "Dirty" and room != self.location
        ]
        if dirty_rooms:
            destination = random.choice(dirty_rooms)
            self.move(destination)
            return f"Moving to room {destination}."

        return "All rooms are clean. Agent is idle."


def ask_room_condition(room):
    """Ask for a room state and validate the answer."""
    while True:
        answer = input(f"Is room {room} Dirty or Clean? ").strip().lower()
        if answer in ("dirty", "clean"):
            return answer.title()
        print("Please enter Dirty or Clean.")


def ask_room_count():
    """Choose the required two-room task or the optional three-room extension."""
    while True:
        answer = input("Number of rooms (2 or 3) [2]: ").strip()
        if not answer:
            return 2
        if answer in ("2", "3"):
            return int(answer)
        print("Please enter 2 or 3.")


def ask_steps():
    """Read a non-negative number of simulation steps."""
    while True:
        answer = input("How many steps should the agent run? ").strip()
        try:
            steps = int(answer)
            if steps >= 0:
                return steps
```

Python source code (continued)

```
        except ValueError:
            pass
        print("Please enter a whole number zero or greater.")


def show_environment(environment, agent):
    """Print a text-based room grid and the complete room-state dictionary."""
    cells = []
    for room, condition in environment.items():
        label = f"{room}-Agent" if room == agent.location else f"{room}-{condition}"
        cells.append(f"[{label}]")
    print(" ".join(cells))
    print(f"Environment state: {environment}")


def main():
    print("Rule-Based Room Cleaning Agent")
    room_count = ask_room_count()
    room_names = ["A", "B", "C"][:room_count]
    environment = {room: ask_room_condition(room) for room in room_names}
    steps = ask_steps()

    # Fixed seed makes the random destination choice reproducible.
    random.seed(7)
    agent = CleaningAgent(environment, location="A")
    print("\nInitial environment:")
    show_environment(environment, agent)

    for step_number in range(1, steps + 1):
        print(f"\nStep {step_number}: Agent is in room {agent.location}")
        action = agent.perceive_and_act()
        print(action)
        show_environment(environment, agent)
        if action == "All rooms are clean. Agent is idle.":
            break

    print("\nSimulation complete.")
    print(f"Final location: room {agent.location}")
    print(f"Final environment: {environment}")


if __name__ == "__main__":
    main()
```